Business Rules for Rental Property Management System (Version 1.0)

1. User Identity & Role Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-USR-01	The system shall recognise exactly four user roles: Owner (Landlord), Manager, Tenant, and Technical Staff. No other roles are valid.	Fact	Static	- SRS 2.2 - backend/routes/auth.js — allowedRoles array
BR-USR-02	A newly registered Tenant account shall be assigned the status pending and must not be permitted to log in until an Owner or Manager explicitly sets the status to active.	Constraint	Static	- SRS 3.1 - backend/routes/auth.js — /register handler; - backend/routes/tenants.js — PUT /:id/status
BR-USR-03	A Tenant whose account status is inactive shall be denied login with the message "Tài khoản của bạn đã bị khóa."	Constraint	Static	backend/routes/auth.js — /login handler
BR-USR-04	An Owner role possesses full access to all system functions including room management, contract creation, invoice generation, financial reporting, and legal report export.	Fact	Static	SRS 2.2
BR-USR-05	A Manager role is granted intermediate permissions: recording meter readings, confirming payments, updating room status, and assigning maintenance requests. A Manager may not create or terminate lease contracts.	Constraint	Static	SRS 2.2, 3.10
BR-USR-06	A Tenant role is restricted to read-only access on data pertaining exclusively to their own room and personal information. Tenants may submit maintenance requests and initiate payments, but may not view or modify data belonging to other tenants or rooms.	Constraint	Static	- SRS 2.2, 6.3 - backend/routes/maintenance-requests.js
BR-USR-07	A Technical Staff role may only view maintenance requests assigned to them, manage repair schedules, and update resolution status. Technical Staff may not access billing, contracts, or tenant personal data.	Constraint	Static	SRS 2.2, 3.10
BR-USR-08	Every user record must store the following mandatory identity fields: fullName, email, password, and role. For Tenant accounts, the system shall additionally capture	Constraint	Static	- SRS §3.9 - backend/database.js — users table schema

	citizenID, phoneNumber, dateOfBirth, gender, permanentAddress, and temporaryAddress.information require 256-bit encryption.			
BR-USR-09	A user's email address must be unique across the entire system. Duplicate email registration shall be rejected with HTTP 409.	Constraint	Static	- backend/routes/auth.js - backend/database.js — UNIQUE constraint on users.email
BR-USR-10	Passwords must be between 8 and 72 characters in length and must contain at least one alphabetic character and one numeric digit.	Constraint	Static	backend/routes/auth.js — isStrongPassword() function
BR-USR-11	All stored passwords must be hashed using bcrypt with a salt round of 10 before persistence. Plain-text passwords shall never be stored.	Constraint	Static	- backend/routes/auth.js - backend/routes/tenants.js
BR-USR-12	Phone numbers, when provided, must conform to the pattern +?[0-9]{8,15}. Invalid phone numbers shall be rejected at registration.	Constraint	Static	backend/routes/auth.js — phone validation regex

2. Authentication & Session Security Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-SEC-01	All authenticated API calls must include a valid JWT Access Token in the Authorization: Bearer <token> header. Requests without a token shall receive HTTP 401.	Constraint	Static	backend/middleware/auth.js — authenticateToken()
BR-SEC-02	The Access Token shall expire in 15 minutes (ACCESS_TOKEN_EXPIRES_IN = '15m').	Constraint	Dynamic	backend/config/auth.js — configurable via JWT_ACCESS_EXPIRES_IN environment variable
BR-SEC-03	The Refresh Token shall expire in 7 days (REFRESH_TOKEN_EXPIRES_IN = '7d').	Constraint	Dynamic	backend/config/auth.js — configurable via JWT_REFRESH_EXPIRES_IN environment variable
BR-SEC-04	Upon successful token refresh, the consumed Refresh Token must be immediately revoked (marked revoked = 1) and a new Refresh Token must be issued. This enforces Refresh Token Rotation and prevents token replay attacks.	Constraint	Static	backend/routes/auth.js — /refresh handler

BR-SEC-05	Upon logout, the system shall insert the Access Token's jti into the revoked_access_tokens table and revoke the Refresh Token in the refresh_tokens table. Subsequent requests bearing the revoked Access Token shall receive HTTP 403.	Constraint	Static	backend/routes/auth.js — /logout handler; backend/middleware/auth.js
BR-SEC-06	Password reset must be performed via a One-Time Password (OTP) delivered to the user's registered email address. The system shall interface with an external Mail/OTP Service (UC-11).	Constraint	Static	- SRS 3.11, 5.2 - SRS Figure A.1.3
BR-SEC-07	The system must purge expired entries from revoked_access_tokens and stale revoked entries from refresh_tokens (older than 7 days post-revocation) via an automated background cleanup job running every 60 minutes.	Constraint	Static	backend/database.js — tokenCleanupInterval
BR-SEC-08	All online payment transaction data and payment history records must be encrypted at rest.	Constraint	Static	SRS 6.3
BR-SEC-09	The JWT_SECRET environment variable is mandatory for server startup. If absent, the server process must terminate with a descriptive error message.	Constraint	Static	backend/server.js — assertAuthConfig(); backend/config/auth.js
BR-SEC-10	The authentication API (/api/auth/*) must be subject to a rate limit of 20 requests per 15-minute window per IP address to prevent brute-force attacks.	Constraint	Dynamic	backend/server.js — authLimiter configuration
BR-SEC-11	All general API endpoints (/api/*) must be subject to a rate limit of 200 requests per 15-minute window per IP address.	Constraint	Dynamic	backend/server.js — apiLimiter configuration

3. Room Management Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-RM-01	Each room must have a unique name within the scope of a single landlord's property. Duplicate room names belonging to the same landlord shall be rejected.	Constraint	Static	backend/routes/rooms.js — POST / and PUT /:id duplicate-name check
BR-RM-02	A room's status must be one of exactly five valid values: **available**, **occupied**, **maintenance**, **reserved**, or **cleaning**. Any other status value shall be rejected or defaulted to available.	Constraint	Static	- backend/routes/rooms.js — VALID_ROOM_STATUSES array - SRS 3.7

BR-RM-03	The state transition rules for room status are as follows: Available → Reserved (triggered by deposit payment); Reserved → Occupied (triggered by contract activation); Occupied → Under Maintenance (triggered by issue report); Occupied → Cleaning (triggered by tenant move-out); Cleaning → Available (triggered by cleaning completion); Under Maintenance → Occupied (triggered by repair completion).	Constraint	Static	SRS Figure A.3.1
BR-RM-04	A room may only be assigned to an active lease contract if its current status is exactly **available**. Attempting to create a contract for an occupied, reserved, or maintenance room shall return an error.	Constraint	Static	backend/routes/contracts.js — POST / status check
BR-RM-05	When a lease contract is created successfully, the associated room's status must be automatically updated to **occupied** within the same database transaction.	Constraint	Static	backend/routes/contracts.js — transaction block
BR-RM-06	When a lease contract is terminated, the associated room's status must be automatically updated to **available** within the same database transaction.	Constraint	Static	backend/routes/contracts.js — PUT /:id/terminate
BR-RM-07	When a room is deleted, all associated invoices and meter readings linked to that room must also be deleted in a cascading manner within a single atomic database transaction.	Constraint	Static	backend/routes/rooms.js — DELETE /:id cascade logic
BR-RM-08	Room price must be a non-negative numeric value. A room price less than zero shall be rejected with an appropriate error message.	Constraint	Static	backend/routes/rooms.js — price validation
BR-RM-09	The system shall display room occupancy statistics including: total rooms, occupied rooms, vacant rooms, and occupancy rate (%).	Fact	Static	- SRS §3.7 - backend/routes/landlord.js — GET /financial-stats

4. Lease Contract Rules

ID	*Rule Definition*	*Type of Rule*	*Static or Dynamic*	*Source*
BR-CON-01	A Tenant may hold a maximum of one active lease contract at any given time. Attempting to create a second active contract for a Tenant who already has one shall be rejected.	Constraint	Static	backend/routes/contracts.js — existingContract check

BR-CON-02	A lease contract must store the following mandatory fields: tenantID, roomID, startDate, endDate, deposit, and rentalPrice. All fields are required; missing fields shall result in rejection.	Constraint	Static	- backend/routes/contracts.js — validation block - SRS 3.8
BR-CON-03	The contract end date must be strictly later than the contract start date. Contracts where endDate <= startDate shall be rejected.	Constraint	Static	backend/routes/contracts.js — date comparison validation
BR-CON-04	Deposit amount must be a non-negative number. Rental price must be a strictly positive number (greater than zero).	Constraint	Static	backend/routes/contracts.js — numeric validation
BR-CON-05	A lease contract may store optional per-unit pricing for electricity (electricity_price) and water (water_price), which will be used to synchronise billing rates at invoice creation time.	Fact	Static	- backend/routes/contracts.js - backend/routes/invoices.js
BR-CON-06	The valid lifecycle states for a contract are: Draft → Pending Signature → Active → Expired or Terminated Early. A contract transitions to Active when the tenant signs and the deposit is paid.	Fact	Static	- SRS 3.8 - SRS Figure A.3.2
BR-CON-07	When a contract reaches its endDate, the system shall automatically flag it as is_expired = 1. The system shall generate automated expiry warning alerts prior to the end date.	Constraint	Static	- SRS 3.8 - backend/database.js — is_expired column
BR-CON-08	During a Room Transfer, the system must finalise the old room's electricity and water meter readings and initialise meter readings for the new room before updating the Tenant-Room link. This must occur atomically.	Constraint	Static	- SRS 3.8 - SRS Figure A.1.10
BR-CON-09	Legal compliance requires the system to store the Tenant's citizenID, dateOfBirth, hometown (permanentAddress), and contract start_date (move-in date) to support export of Temporary Residence Reports for local authorities.	Constraint	Static	- SRS 2.5, 3.14 - backend/routes/reports.js

5. Meter Reading Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-MTR-01	For any meter reading entry after the first, the new electricity index must be greater than or equal to the previous electricity	Constraint	Static	backend/routes/meter-readings.js — index regression check

	index. A value lower than the prior reading shall be rejected with error code INVALID_INDEX_VALUES.			
BR-MTR-02	For any meter reading entry after the first, the new water index must be greater than or equal to the previous water index. A value lower than the prior reading shall be rejected with error code INVALID_INDEX_VALUES.	Constraint	Static	backend/routes/meter-readings.js — index regression check
BR-MTR-03	When creating a monthly invoice, the system must automatically retrieve the previous month's last recorded electricity and water index as the current billing period's starting index. This is the "Previous Meter Reading Auto-Fill" rule.	Computation	Static	- SRS 3.2 - backend/services/meterReadingService.js — getPreviousMeterReading() - backend/routes/landlord.js — GET /rooms/:roomID/previous-reading
BR-MTR-04	MTR-04Electricity consumption for a billing period is calculated as: electricityUsage = currentElectricityIndex − prevElectricityIndex. The result must be non-negative.	Computation	Static	backend/services/invoiceCalculator.js — calculateInvoiceTotal()
BR-MTR-05	Water consumption for a billing period is calculated as: waterUsage = currentWaterIndex − prevWaterIndex. The result must be non-negative.	Computation	Static	backend/services/invoiceCalculator.js — calculateInvoiceTotal()
BR-MTR-06	Both electricityIndex and waterIndex submitted for a meter reading must be finite, non-negative real numbers. Negative or non-numeric values shall be rejected.	Constraint	Static	backend/routes/meter-readings.js — input validation
BR-MTR-07	A meter reading record shall store the previous period's indices (prev_electricity_index, prev_water_index) at the time of insertion for audit traceability.	Constraint	Static	- backend/routes/meter-readings.js - backend/database.js — meter_readings schema

6. Invoice & Billing Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-INV-01	The total invoice amount is computed as: totalAmount = roomPrice + electricityAmount + waterAmount + serviceAmount, where serviceAmount = wifiFee + trashFee.	Computation	Static	backend/services/invoiceCalculator.js — calculateInvoiceTotal()

BR-INV-02	The electricity charge for a billing period is computed as: electricityAmount = electricityUsage × electricityUnitPrice.	Computation	Dynamic	- backend/services/invoiceCalculator.js - unit price is set by Owner via service configuration
BR-INV-03	The water charge for a billing period is computed as: waterAmount = waterUsage × waterUnitPrice.	Computation	Dynamic	- backend/services/invoiceCalculator.js - unit price is set by Owner via service configuration
BR-INV-04	The system must prevent creation of a duplicate utility invoice (one containing electricity or water charges) for the same room in the same calendar month and year. Duplicate utility bill creation shall return error code DUPLICATE_UTILITY_BILL.	Constraint	Static	backend/routes/invoices.js — isUtilityBill() and duplicate check query
BR-INV-05	An invoice's billing period (month/year) must not pre-date the associated lease contract's start date. Invoices for periods before contract commencement shall be rejected with error code BILLING_PERIOD_INVALID.	Constraint	Static	backend/routes/invoices.js — contract start date comparison
BR-INV-06	A newly created invoice must have the status unpaid and payment status Unpaid. An invoice may transition through the following payment states: unpaid → partial → paid.	Fact	Static	- backend/routes/invoices.js - backend/database.js — invoices schema
BR-INV-07	An invoice that has been fully paid (status = paid) must not be deleted. Attempting to delete a paid invoice shall return error code CANNOT_DELETE_PAID_INVOICE.	Constraint	Static	backend/routes/invoices.js — DELETE /:id guard
BR-INV-08	When an invoice is deleted, all associated meter readings for the same room and billing period must have their electricity and water indices reset to zero (0), and their invoice_id foreign key must be set to NULL.	Constraint	Static	backend/routes/invoices.js — DELETE /:id cleanup logic
BR-INV-09	All invoice amounts (room, electricity, water, service, total) must be finite, non-negative numbers. Any amount failing this check must cause the invoice calculation to raise a BillingValidationError.	Constraint	Static	backend/services/invoiceCalculator.js — validateNonNegativeNumber()
BR-INV-10	Service unit prices (electricityUnitPrice, waterUnitPrice) must be finite, non-negative numbers configured by the Owner. Missing or invalid unit prices must raise a ServicePricingConfigError and prevent invoice creation.	Constraint	Dynamic	backend/services/invoiceCalculator.js — missingServicePricing check

BR-INV-11	The system must support partial payment recording. A payment of amount less than total_amount shall update paid_amount to the cumulative sum and set status = 'partial'. Full payment completion sets status = 'paid'.	Constraint	Static	backend/routes/invoices.js — POST /mock-payment/confirm
BR-INV-12	All generated invoices must be exportable as PDF files upon confirmation.	Constraint	Static	- SRS 3.2 - backend/routes/invoices.js — exportPDF() reference - backend/routes/reports.js
BR-INV-13	The system shall send automated payment reminder notifications to Tenants for invoices that are unpaid or overdue.	Constraint	Static	- SRS 3.6, 3.13 - frontend/src/components/NotificationBell.jsx — overdue detection

7. Payment Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-PAY-01	Accepted payment methods include: Cash (Tiền mặt), Bank Transfer (Chuyển khoản), MoMo e-wallet, and ZaloPay e-wallet. The system shall also support Bank QR Code integration via third-party payment APIs.	Fact	Dynamic	- SRS 3.3, 5.2 - frontend/src/pages/tenant/TenantInvoices.jsx
BR-PAY-02	A payment_method field is mandatory when recording a payment. Invoices marked as paid without specifying a payment method shall be rejected.	Constraint	Static	backend/routes/invoices.js — PUT /:id/pay validation
BR-PAY-03	Attempting to pay or mark as paid an invoice that already has status = 'paid' shall be rejected with error code ALREADY_PAID.	Constraint	Static	backend/routes/invoices.js — PUT /:id/pay and /mock-payment/confirm
BR-PAY-04	Upon successful online payment via QR/e-wallet, the system must receive a webhook notification from the Payment API and update the invoice's payment status automatically.	Constraint	Static	- SRS 5.2 - SRS Figure A.1.11
BR-PAY-05	The remaining amount due for a partially-paid invoice is computed as: remainingAmount = MAX(0, totalAmount − paidAmount).	Computation	Static	backend/routes/invoices.js — GET /mock-payment/:id
BR-PAY-06	A Manager may confirm payments supporting both partial and full payment modes. Partial payments must update	Computation	Static	- SRS 3.10 - backend/routes/invoices.js

	paid_amount without changing the status to paid until paid_amount >= total_amount.			

8. Maintenance Request Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-MNT-01	A maintenance request must include a non-empty description field. Submissions without a description or with an empty-string description shall be blocked.	Constraint	Static	- SRS 3.4 - backend/routes/maintenance-requests.js — POST validation - backend/routes/tenants.js
BR-MNT-02	The system shall automatically assign High Priority to any maintenance request whose description contains one or more of the following keywords: gas, điện giật (electric shock), cháy (fire), nổ (explosion), nguy hiểm (dangerous), khẩn cấp (urgent), rò rỉ (leak). This implements safety requirement SAF-2.	Computation	Static	- SRS 6.4 SAF-2 - backend/routes/tenants.js — highPriorityKeywords array and auto-priority logic
BR-MNT-03	Maintenance requests classified as High Priority shall immediately trigger a notification to the Owner/Manager.	Constraint	Static	- SRS 6.4 SAF-2 - SRS Figure A.1.9
BR-MNT-04	A maintenance request may only be updated (status changed, staff assigned, cost recorded) by an Owner or Manager role. A Tenant may only update the description and category of their own requests, and only while the request status is still pending.	Constraint	Static	backend/routes/maintenance-requests.js — role checks in PUT /:id
BR-MNT-05	A Tenant may only delete their own maintenance request, and only while its status is pending. Requests that have been accepted (status in_progress, completed, or cancelled) may not be deleted by the Tenant.	Constraint	Static	backend/routes/maintenance-requests.js — DELETE /:id guard
BR-MNT-06	The system shall store the cost of each maintenance or repair job. The total of all maintenance costs shall be used to compute net profit in financial reporting: netProfit = totalRevenue − totalMaintenanceCost.	Computation	Static	- SRS 3.12 - backend/routes/landlord.js — GET /financial-stats
BR-MNT-07	The valid status lifecycle for a maintenance request is: pending → in_progress → completed or cancelled.	Fact	Static	- SRS 3.10 - backend/routes/maintenance-requests.js

9. Notification Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-NOT-01	The system shall dispatch automated notifications for the following trigger events: (a) new invoice generated, (b) invoice payment overdue, (c) lease contract expiring within 30 days, (d) maintenance request status change, (e) new tenant registration pending approval.	Constraint	Static	- SRS §3.13 - frontend/src/components/NotificationBell.jsx
BR-NOT-02	A notification broadcast by an Owner may target one of four recipient scopes: all_tenants (all tenants with active contracts), room (all tenants in a specific room), tenant (a single specific tenant), or selected_tenants (a custom list of tenant IDs).	Fact	Static	backend/routes/notifications.js — recipientType logic
BR-NOT-03	An Owner may only send notifications to Tenants who have an active lease contract within their managed properties. Attempting to send to a tenant without an active contract shall be rejected.	Constraint	Static	backend/routes/notifications.js — active contract verification

10. Financial & Legal Reporting Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-RPT-01	The system shall calculate and display revenue aggregated by month, quarter, and year.	Fact	Static	- SRS 3.12 - backend/routes/landlord.js — GET /financial-stats
BR-RPT-02	Bad debt is defined as the sum of (total_amount − paid_amount) for all invoices with status IN ('unpaid', 'partial').	Computation	Static	- SRS 3.12 - backend/routes/landlord.js — GET /financial-stats bad debt query
BR-RPT-03	The Temporary Residence Report exported for local police must include, for every active-contract Tenant: fullName, citizenID, dateOfBirth, permanentAddress, and contract startDate (move-in date). The document must be formatted to Vietnamese government standards and exported as a PDF file.	Constraint	Static	- SRS 3.14 - backend/routes/reports.js — GET /temporary-residence
BR-RPT-04	The Tax Declaration Report must be exportable as an XLSX (Excel) file and must include, for each paid invoice: billing period (month/year), room name, rent amount,	Constraint	Static	- SRS 3.14 - backend/routes/reports.js — GET /tax

	electricity amount, water amount, service amount, total amount, and a grand total row.			
BR-RPT-05	The Tax Report must only include invoices with status = 'paid' or payment_status = 'Paid'. Unpaid or partial-payment invoices must be excluded.	Constraint	Static	backend/routes/reports.js — SQL WHERE clause

11. Data Integrity & Audit Rules

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-AUD-01	The system must maintain an **audit log** (audit_logs table) recording all significant mutation actions including: price changes for rooms, contract creation and deletion, invoice creation and deletion, and payment confirmations. Each log entry must capture: userId, action, entityType, entityId, description, oldValues, newValues, status, and createdAt.	Constraint	Static	- SRS 4.4, 6.3 - backend/services/auditLog.js - backend/middleware/audit.js
BR-AUD-02	HTTP PUT and DELETE operations on the rooms and lease_contracts tables must be automatically intercepted by the auditMutations middleware and logged before the response is finalised.	Constraint	Static	backend/middleware/audit.js — auditMutations() middleware
BR-AUD-03	The system must perform **automatic daily backups** to ensure full data recovery in the event of system failure.	Constraint	Static	SRS 4.4, 6.6
BR-AUD-04	The system response time for any user interaction must be **less than 4.5 seconds** under normal operating conditions.	Constraint	Static	SRS 2.4, 6.2
BR-AUD-05	The system must operate with **24/7 availability** to ensure tenants can submit emergency requests and pay invoices at any time.	Constraint	Static	SRS 6.5
BR-AUD-06	All DELETE operations on databases must enforce referential integrity. Before deleting a parent record (e.g., Room), all dependent child records (Invoices, MeterReadings) must be removed within the same atomic transaction.	Constraint	Static	- backend/routes/rooms.js — DELETE /:id cascade - backend/database.js — PRAGMA foreign_keys = ON
BR-AUD-07	The system must store **emergency contact information** for all Tenants, accessible by the Owner/Manager in the event of a facility emergency. This implements safety requirement SAF-1.	Constraint	Static	SRS 6.4 SAF-1